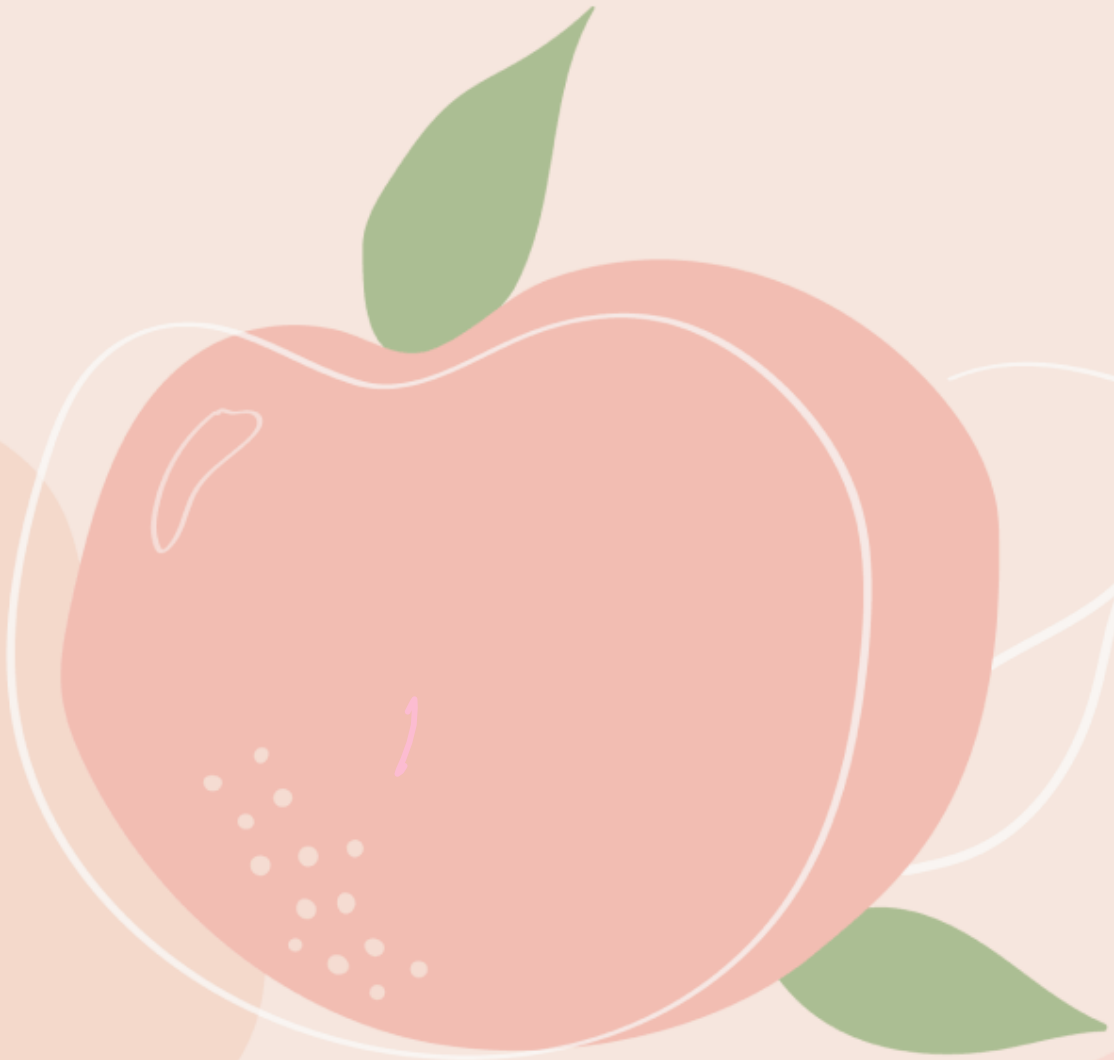
Perception- Deep Learning



Deep Learning: subfield of ML that uses algorithms inspired by the structure and fxn of neural network.

↳ it analyze data, learn from that data & then make predictions about new data.

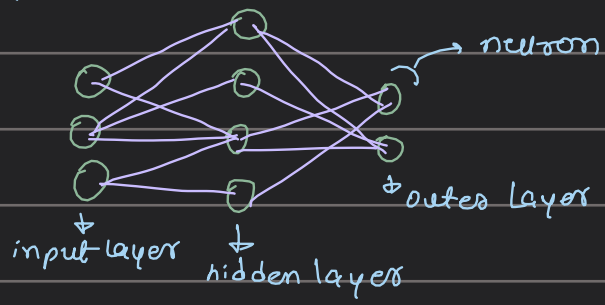
↳ Supervised, unsupervised, semi-supervised

Artificial Neural Networks (ANNs)

↳ computer systems inspired by brain's neural network

↳ each connection can transmit a signal from one neuron to other, processing the signal & passing it.

↳ consists of:
- input layer
- Hidden layer
- Output layer



Keras sequential model:

↳ a seq. model of linear stack of layers

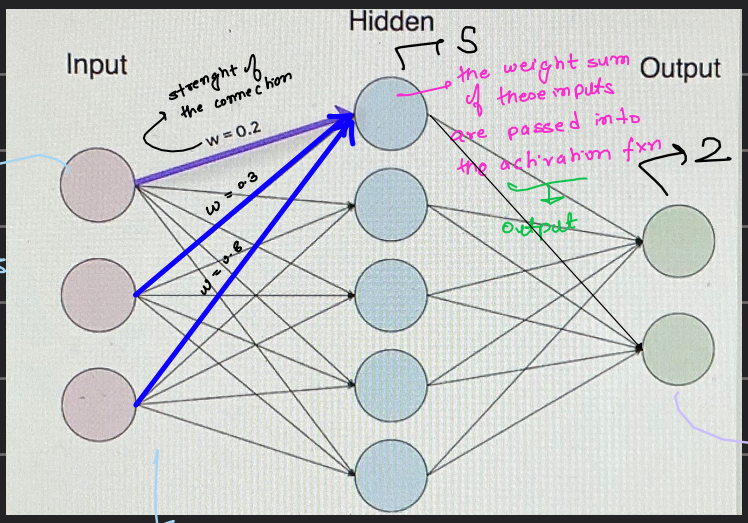
```
[7]: from keras.models import Sequential
     from keras.layers import Dense, Activation

     model = Sequential([
         Dense(5, input_shape=(3,), activation = 'relu'),
         Dense(2, activation = 'softmax')
     ])
```

- first hidden layer
- no. of neurons in each layer
- passing an array into this constructor ('sequential')
- ↳ 3 neurons in the input layer (input-shape)
- Types of Layers: (Hidden layers which connects the inputs to the outputs

↳

Layers in an ANN:
- Dense layer
- Convolutional layer - - - -

① Dense layer:



represents individual features from the dataset passed into the model

each input is connected to the next layer

output = activation (∈ (0,1) (weighted sum)

will be passed into the next neuron

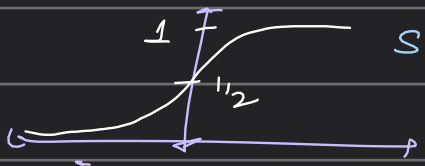
no. of neurons in the output ↳ no. of categories our model will predict (cat/dog)

Activation Function: activation fxn of a neuron defines the output of that neuron given a set of inputs

↓

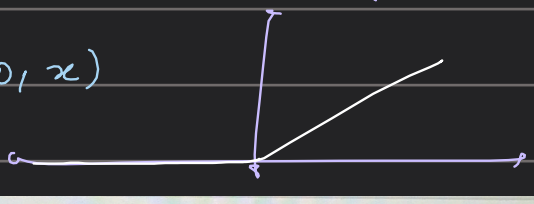
transform the sum to a number b/w so. lower & upper limit

Types: ① sigmoid: $S(x) = \frac{1}{1+e^{-x}}$



S(sum) ∈ (0,1)

② Relu: $f(x) = \max(0, x)$

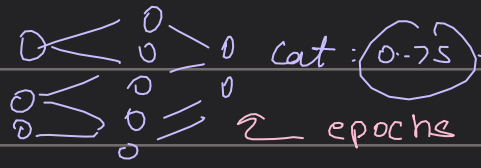


```
model = Sequential()
model.add(Dense(5, input_shape=(3, )))  ↳ first add the layer
model.add(Activation('relu'))  ↳ then add the activation fxn
```

Training the model: ↳ optimizing weights (stochastic Gradient descent)

↳ minimize loss fxn

Loss fxn: cat



cat: 0.75 → (1 − 0.75) → we need to minimize this

↳ 2 epochs

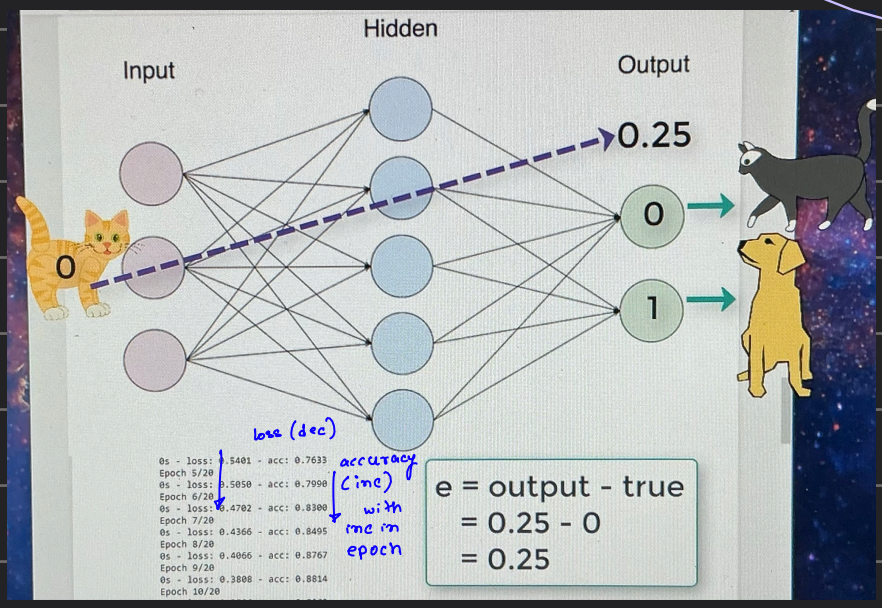
↓

we pass cat again and again by changing the weights fxn ↳ until the loss is minimized

$\Delta = \frac{\partial(\text{loss})}{\partial(\text{weight})} \times \text{learning rate}$

$w_{new} = w - \Delta$

How to determine the loss?



⑦ Adam ↳ SGD ↓ optimizer

Loss fxn: mean sq. error

$mse = \frac{e_1^2 + e_2^2 + \cdots + e_n^2}{n}$

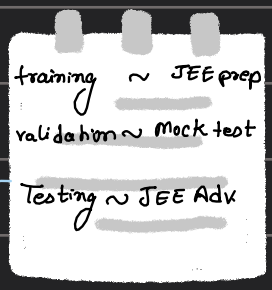
Training Set: the set of data on which the the model is trained
↓ labelled

during each epoch the model will be trained on the same data again & again

Validation Set: a set diff from the training set which is used to validate the prediction of the model.
↳ labelled

ps give info of that may ajust the hyperparameters
↳ just like the training set after every epoch the model will be validating the data in the validation set.
↳ after every epoch the weights in the training set will change but not of the validation set.

to see that the model should not be over/under fitting



test set: ↳ set on which predictions are made

Overfitting: the model hasn't generalised well
↳ model good at predicting the training set & not the testing set.

Validation set accuracy << training set accuracy

↳ inc Dropout (remove some neurons)

Solution: Data Augmentation

eg. for image data: creating new set of data from existing data by croping, resizing, rotated, colour ----

Underfitting: → when the training accuracy is low

soln ↓

inc more layers, more data, change the layer type → Dec Dropout

Supervised Learning: → labled training set

eg: training_set = [[154, 50], [158, 54], [170, 78], [184, 90]]

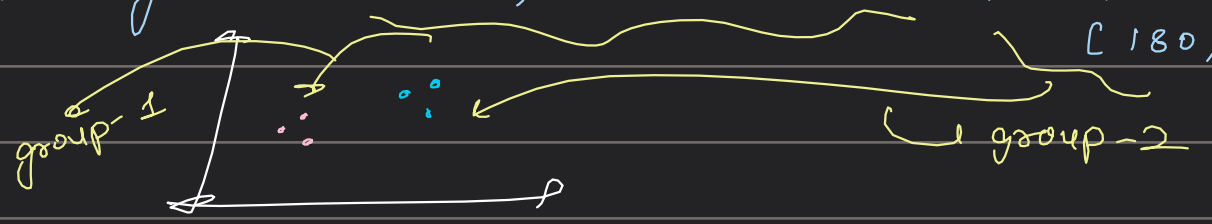
training_label: [1, 1, 0, 0]

labels: 1: female, 0: male

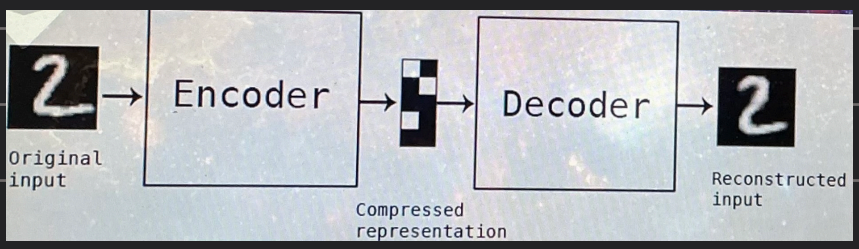
Unsupervised Learning: → unlabled training set (no talk of accuracy)

eg: clustering algorithm:

training_set = [[160, 56], [154, 50], [148, 50], [170, 80], [176, 82], [180, 90]]



eg: Auto encoder: an ANN that takes in an input & outputs a reconstruction of this input. eg: Denoising images



Semi supervised Learning: some data is labelled & some unlabelled

↳ pseudo supervised: train the data on a labelled dataset from that trained model predict new data & put the

predicted label on the data → now we train the model on the dataset which was truly labelled & pseudo labelled

One-hot Encoding: changing the labels to vectors of 0, 1, --

object-1 - [1 0 0 0]

object-2 - [0 1 0 0]

object-3 - [0 0 1 0]

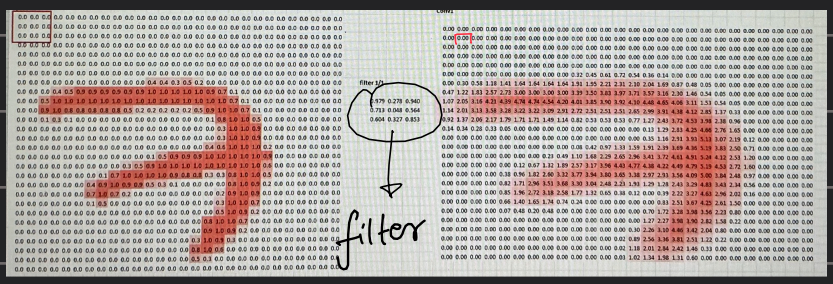
object-4 - [0 0 0 1]

Convolutional Neural Networks (CNNs) — image analysis

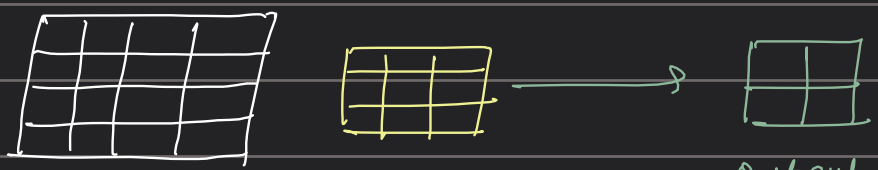
↓

Hidden layers ≡ Convolutional layer — consists of a filter — detect pattern, classification

these layers can detect patterns like edges, colour, texture, ----



Zero Padding:



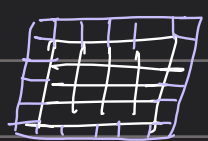
4×4 reduced to 2×2

n×n : input

f×f : filter

output : (n-f+1)×(n-f+1)

in order to get 4×4 as an output → we use Zero Padding → add layer of zeroes around the input to prevent dec in size of output



Zero Padding

```
]: import keras
   from keras.models import Sequential
   from keras.layers import Activation
   from keras.layers.core import Dense, Flatten
   from keras.layers.convolutional import *
```

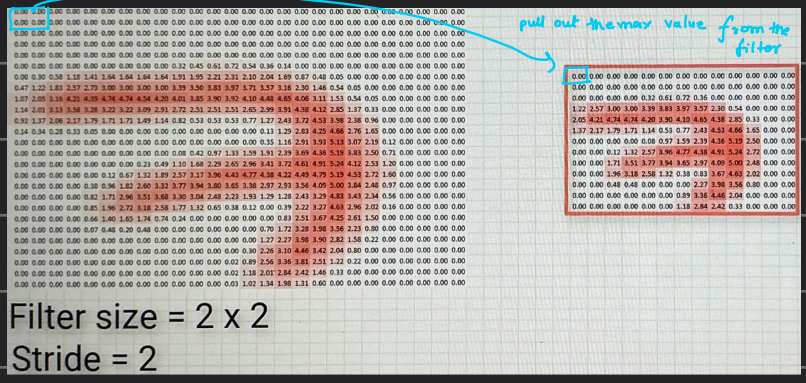
```
]: model_valid = Sequential([
        Dense(16, activation='relu', input_shape=(20,20,3)),
        Conv2D(32, kernel_size=(3, 3), activation='relu', padding='valid'),
        Conv2D(64, kernel_size=(5, 5), activation='relu', padding='valid'),
        Conv2D(128, kernel_size=(7, 7), activation='relu', padding='valid'),
        Flatten(),
        Dense(2, activation='softmax'),
    ])
```

```
]: model_valid.summary()
```

Max Pooling:

↓

dec the size of the input



reduced to half the dimension of the output

```
]: import keras
   from keras.models import Sequential
   from keras.layers import Activation
   from keras.layers.core import Dense, Flatten
   from keras.layers.convolutional import *
   from keras.layers.pooling import *
```

```
]: model = Sequential([
       Dense(16, activation='relu', input_shape=(20,20,3)),
       Conv2D(32, kernel_size=(3, 3), activation='relu', padding='same'),
       MaxPooling2D(pool_size=(2, 2), strides=2, padding='valid'),
       Conv2D(64, kernel_size=(5, 5), activation='relu', padding='same'),
       Flatten(),
       Dense(2, activation='softmax'),
   ])
```

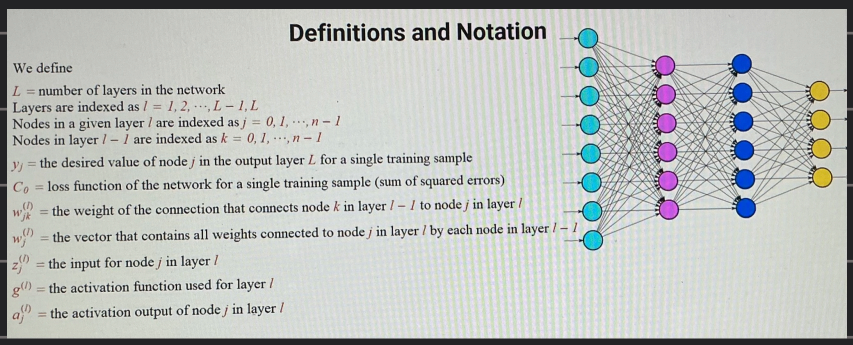
Backpropagation: → the tool which SGD uses to calculate the gradient Descent

SGD: to inc the value for the correct output node & dec for the incorrect ones → dec the loss

↓

the weights of the previous layers

↳ we go in backward direction changing the weights of the previous layers



Loss C_0:

$(a_j^{(L)} - y_j)^2$ is the squared diff of the activation output & the desired output of node j in the output layer L

$(a_j^{(L)} - y_j)^2$: Loss in node j in the output layer L

↓

→ a fxn of Activation output

So the total loss $C_0 = \sum_{j=0}^{n-1} (a_j^{(L)} - y_j)^2$

Input:

input for node j in layer l is the weighted sum of the activation outputs from the previous layer $l-1$

for node j in layer l: $w_{jk}^{(l)} a_k^{(l-1)}$

So, the input for a node j in layer l is $z_j^{(l)} = \sum_{k=0}^{n-1} w_{jk}^{(l)} a_k^{(l-1)}$

Activation Output: of a given node j in layer l is the result of passing the input $z_j^{(l)}$ the activation fxn $g^{(l)}$

the activation output of node j in Layer l is $a_j^{(l)} = g^{(l)}(z_j^{(l)})$

Chain Rule: $C_0(a_j^{(L)})$ $\quad a_j^{(L)} = g^{(L)}(z_j^{(L)})$

the input for node j is a fxn of all weights connected to j

$z_j^{(L)}(w_j^{(L)}) \Rightarrow$ $\boxed{C_{0j} = C_{0j}(a_j^{(L)}(z_j^{(L)}(w_j^{(L)})))}$

the total loss of the network for a single input is also a composition fxn

for differentiation we use chain rule

Calculation of the gradient: derivative of the loss wrt the weights

$w_{12}^{(L)}$: weight that connects node 2 in layer $L-1$ to node 1 in layer L

since $C_0 = C_0(a_1^{(L)}(z_1^{(L)}(w_{12}^{(L)})))$

So, by chain rule,

$$\frac{\partial C_0}{\partial w_{12}^{(L)}} = \left(\frac{\partial C_0}{\partial a_1^{(L)}}\right)\left(\frac{\partial a_1^{(L)}}{\partial z_1^{(L)}}\right)\left(\frac{\partial z_1^{(L)}}{\partial w_{12}^{(L)}}\right)$$

① $\frac{\partial C_0}{\partial a_1^{(L)}}$, $C_0 = \sum_{j=0}^{n-1} (a_j^{(L)} - y_j)^2$ Layer: L: 5 nodes

$$\frac{\partial C_0}{\partial a_1^{(L)}} = \frac{\partial}{\partial a_1^{(L)}} \left(\sum_{j=0}^{n-1} (a_j^{(L)} - y_j)^2 \right) = \frac{\partial}{\partial a_1^{(L)}} \left((a_0^{(L)} - y_0)^2 + (a_1^{(L)} - y_1)^2 + (a_2^{(L)} - y_2)^2 + (a_3^{(L)} - y_3)^2 \right)$$

$$= \frac{\partial}{\partial a_1^{(L)}} (a_0^{(L)} - y_0)^2 + \frac{\partial}{\partial a_1^{(L)}} (a_1^{(L)} - y_1)^2 + \frac{\partial}{\partial a_1^{(L)}} (a_2^{(L)} - y_2)^2 + \frac{\partial}{\partial a_1^{(L)}} (a_3^{(L)} - y_3)^2$$

$$= 2(a_1^{(L)} - y_1)$$

② $\frac{\partial a_1^{(L)}}{\partial z_1^{(L)}}$

$a_j^{(L)} = g^{(L)}(z_j^{(L)})$ (j=1) $= g^{(L)}(z_1^{(L)}) \Rightarrow \frac{\partial a_j^{(L)}}{\partial z^{(L)}} = g'^{(L)}(z_1^{(L)})$

③ $\frac{\partial z_1^{(L)}}{\partial w_{12}^{(L)}}$, $z_1^{(L)} = \sum_{K=0}^{n-1} w_{jK}^{(L)} a_K^{(L-1)}$

$$\frac{\partial z_1^{(L)}}{\partial w_{12}^{(L)}} = \frac{\partial}{\partial w_{12}^{(L)}} \left(\sum_{K=0}^{n-1} w_{1K}^{(L)} a_K^{(L-1)} \right) = \frac{\partial}{\partial w_{12}^{(L)}} \left(w_{10}^{(L)} a_0^{(L-1)} + w_{11}^{(L)} a_1^{(L-1)} + a_{12}^{(L)} a_2^{(L-1)} + \cdots + a_{15}^{(L)} a_5^{(L-1)} \right)$$

$$= \frac{\partial}{\partial w_{12}^{(L)}} (w_{10}^{(L)} a_0^{(L-1)}) + \frac{\partial}{\partial w_{12}^{(L)}} (w_{11}^{(L)} a_1^{(L-1)}) + \frac{\partial}{\partial w_{12}^{(L)}} (w_{12}^{(L)} a_2^{(L-1)}) + \cdots + \frac{\partial}{\partial w_{12}^{(L)}} (w_{15}^{(L)} a_5^{(L-1)})$$

$$= a_2^{(L-1)}$$

the input for node 1 in layer L will respond to change in the weight $w_{12}^{(L)}$ by an amount equal to the activation output for node 2 in the previous layer L-1.

So, $\frac{\partial C_0}{\partial w_{12}^{(L)}} = 2(a_1^{(L)} - y_1)(g'^{(L)}(z_1^{(L)}))(a_0^{(L-1)})$

↳ wrt to 1 particular weight b/w a pair of particular nodes of particular layers

for $\frac{\partial C}{\partial w_{12}^{(L)}} = \frac{1}{n} \sum_{i=0}^{n-1} \frac{\partial C_i}{\partial w_{12}^{(L)}}$

↳ wrt to 1 particular weight for all n training samples

Derivative of the loss fxn wrt weights not in the output layer

$$\frac{\partial C_0}{\partial w_{22}^{(L-1)}} = \left(\frac{\partial C_0}{\partial a_2^{(L-1)}}\right)\left(\frac{\partial a_2^{(L-1)}}{\partial z_2^{(L-1)}}\right)\left(\frac{\partial z_2^{(L-1)}}{\partial w_{22}^{(L-1)}}\right)$$

$(L-2)$ $(L-1)$

2 → 2

since C_0 is not the direct fxn of $a_j^{(L-1)}$ — only of the activation output of the outer layer

as above

each node j in L, the Loss C_0 depends on $a_j^{(L)}$ & $a_j^{(L)}$ depends on $z_j^{(L)}$

$$\text{so, } \frac{\partial C_0}{\partial a_2^{(L-1)}} = \sum_{j=0}^{n-1}\left(\left(\frac{\partial C_0}{\partial a_j^{(L)}}\right)\left(\frac{\partial a_j^{(L)}}{\partial z_j^{(L)}}\right)\left(\frac{\partial z_j^{(L)}}{\partial a_2^{(L-1)}}\right)\right)$$

as above; $\frac{\partial z_j^{(L)}}{\partial a_2^{(L-1)}} = 0$

$$\frac{\partial z_j^{(L)}}{\partial a_2^{(L-1)}} = \frac{\partial}{\partial a_2^{(L-1)}}\sum_{k=0}^{n-1} w_{jk}^{(L)} a_k^{(L-1)} = \frac{\partial}{\partial a_2^{(L-1)}}\left(w_{j0}^{(L)} a_0^{(L-1)} + \cdots + w_{j5}^{(L)} a_5^{(L-1)}\right)$$

$$= \frac{\partial}{\partial a_2^{(L-1)}} w_{j0}^{(L)} a_0^{(L-1)} + \frac{\partial}{\partial a_2^{(L-1)}} w_{j1}^{(L)} a_1^{(L-1)} + \frac{\partial}{\partial a_2^{(L-1)}} w_{j2}^{(L)} a_2^{(L-1)} + \cdots + \frac{\partial}{\partial a_2^{(L-1)}} w_{j5}^{(L)} a_5^{(L-1)}$$

(each term $\to 0$ except the $w_{j2}^{(L)}$ term)

$$= w_{j2}^{(L)}$$

the input for any node j in layer L will respond to a change in $a_2^{(L-1)}$ by an amount equal to the weight connecting node 2 in layer $L-1$ to node in layer L.

$$\text{so, } \frac{\partial C_0}{\partial a_2^{(L-1)}} = \sum_{j=0}^{n-1}\left(2\left(a_j^{(L)} - y_j\right)\left(g'^{(L)}(z_j^{(L)})\right)\left(w_{j2}^{(L)}\right)\right)$$

$$\text{so, } \frac{\partial C_0}{\partial w_{22}^{(L-1)}} = \left(\sum_{j=0}^{n-1}\left(2\left(a_j^{(L)} - y_j\right)\left(g'^{(L)}(z_j^{(L)})\right)\left(w_{j2}^{(L)}\right)\right)\right)\left(g'^{(L-1)}(z_2^{(L-1)})\right)\left(a_2^{(L-2)}\right)$$

So for all the n training samples

$$\frac{\partial C}{\partial a_2^{(L-1)}} = \frac{1}{n}\sum_{i=0}^{n-1}\frac{\partial C_i}{\partial a_2^{(L-1)}}$$

Vanishing gradient problem:

very small gradient ⇒ change in weights is really small

weight becomes stuck instead of reaching its optimal value

this is more prominant in early layers : more nodes

gradient = $a * b * c * \cdots * x$ $\quad a, b, \ldots x < 0$

gradient quickly becomes small

Exploding gradient problem:

very large gradient ⇒ change in weights is really big

we might even overpass the optimal value

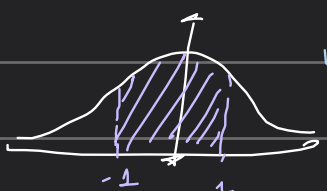
this is more prominant in early layers : more nodes

gradient = $a * b * c * \cdots * x$ $\quad a, b, \ldots x > 0$

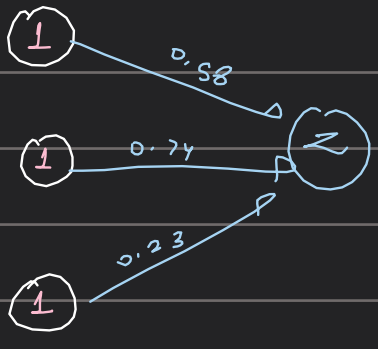
gradient quickly becomes large

How are weights initialised?

↳ randomly generated from a normal distribution



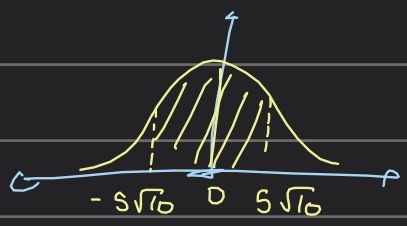
with $\mu = 0, \sigma = 1$



we be normally distributed around 0

$z = 1 \times 0.58 + 1 \times 0.24 + \cdots$

$var(z) = 250$, $std = \sqrt{250} = 5\sqrt{10}$



But large variance can be a problem so, → How to reduce it?

↓

By reducing the variance of the weights

Xavier Initialization :

var (weights) $= 2/n$ weights $* \sqrt{2/n}$

Weight Initialization

```
from keras.models import Sequential
from keras.layers import Dense, Activation
```

```
model = Sequential([
    Dense(16, input_shape=(1,5), activation='relu'),
    Dense(32, activation='relu', kernel_initializer='glorot_uniform'),
    Dense(2, activation='softmax')
])
```

Bias in ANN :

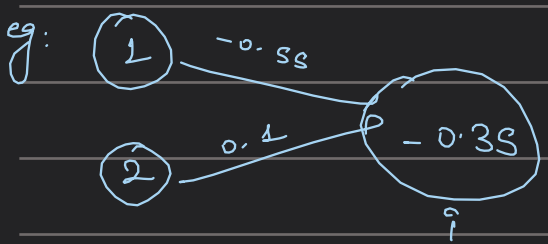
* each neuron has a bias
* Biases are learnable

↓

like a threshold which determines if a neuron will transmit or not

$$a_j^{(L)} = g^{(L)}\left(w_1^{(L)} a_1^{(L)} + w_2^{(L)} a_2^{(L)} + \cdots + w_n^{(L)} a_n^{(L)} + b\right)$$

eg:



$g^{(L)}(x) = \max(x, 0)$

$z_i^{(L)} = 1 \times -0.58 + 2 \times 0.1 = -0.35$

$g^{(L)}(x) = \max(x, -0.35) = 0$

↳ the neuron is inactive

for $g(x) = -0.35$

0 → -1 ↳ b = opposite of -1

b = 1

$z_i^{(L)} = -0.35 + 1$

$= 0.65$ ↳ $g^{(L)}(z_i^{(L)}) = 0.65 = a_i^{(L)}$

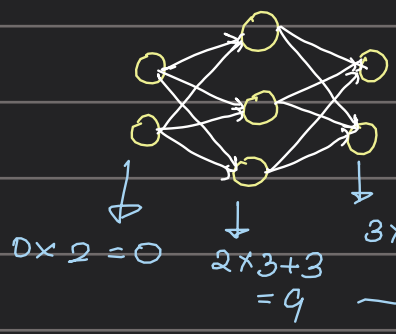
eg: so if we want the neuron to be active when its value is ≥ 5 $\Rightarrow$ $b = -5$

Learnable Parameter: A parameter that is learned by the network during training eg: bias, weights

(eg: Dense)

$$\sum_{i=0}^{n-1} (\text{no. of inputs}) * (\text{no. of outputs}) + \text{bias}$$

(no. of outputs $\rightarrow$ no. of nodes; no. of inputs $\rightarrow$ weights)



17 Learnable parameters

$\rightarrow$ it has filters (Kernals) as extra parameter

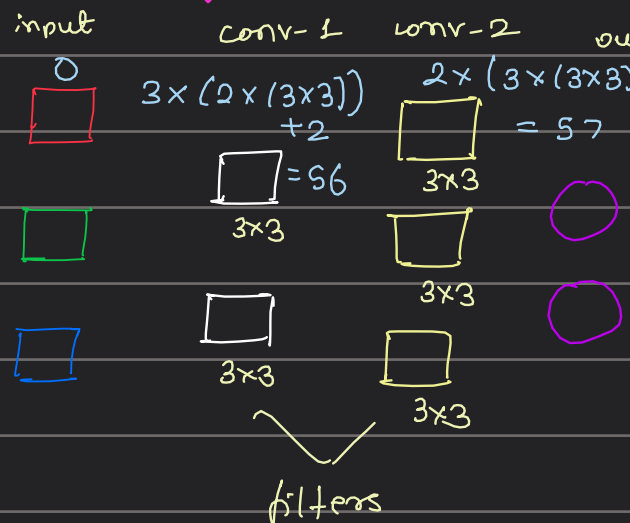
for CNN:

$$\sum_{i=0}^{n-1} (\text{no. of inputs}) * (\text{no. of outputs}) + \text{bias}$$

(no. of outputs $\rightarrow$ (no. of filters) * (size of filter); bias $\rightarrow$ no. of filters; no. of inputs $\rightarrow$ weights)

if last layer is dense: input = no. of nodes

if last layer is conv.: input = no. of nodes



$(20 \times 20) \times 3 \times 2 + 2 = 2402$

$\downarrow$

size of the input layer

$2402 + 56 + 57 = 2515$

Regularization: to prevent overfitting or reducing var. to dec complexity
↳ loss + x ↳ penelises for extreamly large weights

L2 Regularization: $x = \left(\sum_{j=1}^{n} \|w^{[j]}\|^2\right)\frac{\lambda}{2m}$

n: no. of layers $w^{[j]}$ = the weight matrix of the j^{th} layer
m: no. of inputs λ: regularization parameter

So if we keep λ large (weights) will be very small since loss ↳ min

By this some layers might be neglected ⟶ reduced complexity

Regularization

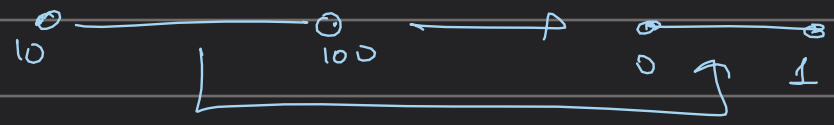
```
import keras
from keras import backend as K
from keras.models import Sequential
from keras.layers import Activation
from keras.layers.core import Dense
from keras import regularizers
```

```
model = Sequential([
    Dense(16, input_shape=(1,), activation='relu'),
    Dense(32, activation='relu', kernel_regularizer=regularizers.l2(0.01)),
    Dense(2, activation='sigmoid')
])
```

Fine-tunning / Trans for Learning: ↳ to use a model which has already been trained before & we can use it for a similar classification

↓ import change pre trained model & certain layers (generally that later ones)

Batch Normalization:



eg: when one weights becomes really big compared to others

(1) Normalize the output from $g^{(L)}$, $z = \frac{x-m}{s}$

(2) $z * a + b$ ← new weight

Batch Normalization

In [6]:
```
from keras.models import Sequential
from keras.layers import Dense, Activation, BatchNormalization
```

In [10]:
```
model = Sequential([
    Dense(16, input_shape=(1,5), activation='relu'),
    Dense(32, activation='relu'),
    BatchNormalization(axis=1),
    Dense(2, activation='softmax')
])
```

In [ ]:
```
#beta_initializer: Initializer for the beta weight.
#gamma_initializer: Initializer for the gamma weight.
```

```
import tensorflow as tf
import numpy as np
import matplotlib.pyplot as plt
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Flatten, Dense, Activation
```

```
(x_train, y_train), (x_test, y_test) = tf.keras.datasets.mnist.load_data()
```

```
gray_scale = 255

x_train = x_train.astype('float32') / gray_scale
x_test = x_test.astype('float32') / gray_scale
```

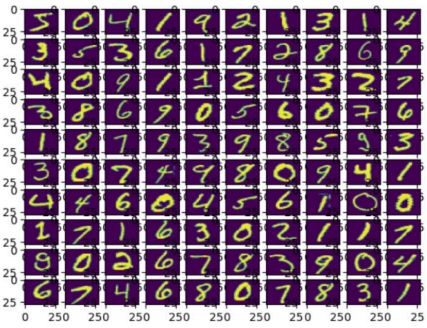
```
print("Feature matrix (x_train):", x_train.shape)
print("Target matrix (y_train):", y_train.shape)
print("Feature matrix (x_test):", x_test.shape)
print("Target matrix (y_test):", y_test.shape)
```

```
Feature matrix (x_train): (60000, 28, 28)
Target matrix (y_train): (60000,)
Feature matrix (x_test): (10000, 28, 28)
Target matrix (y_test): (10000,)
```

```
fig, ax = plt.subplots(10, 10)
k = 0
for i in range(10):
    for j in range(10):
        ax[i][j].imshow(x_train[k].reshape(28, 28), aspect='auto')
        k += 1
plt.show()
```

```
from tensorflow.keras.layers import Flatten, Dense, Activation, Input

model = Sequential([
    Input(shape=(28, 28)),        # ← new line
    Flatten(),                     # ← no input_shape here
    Dense(256, activation='sigmoid'),
    Dense(128, activation='sigmoid'),
    Dense(10, activation='softmax'),
])
```

```
from tensorflow.keras.optimizers import Adam

model.compile(optimizer=Adam(learning_rate=0.0001), loss='sparse_categorical_crossentropy', metrics=['accuracy'])
```

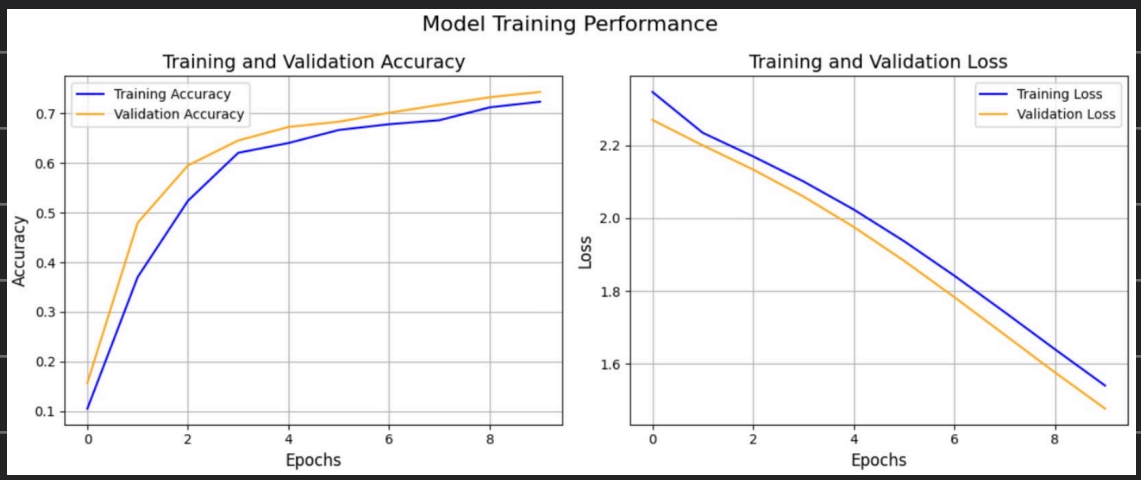


```
mod = model.fit(x_train, y_train, epochs=10,batch_size=2000,validation_split=0.2)

print(mod)
```

batch_size ↷ no. of samples that will be passed through to the network at one time

```
Epoch 1/10
24/24 ━━━━━━━━━━━━━━━━━━━━ 21s 266ms/step - accuracy: 0.1049 - loss: 2.3467 - val_accuracy: 0.1567 - val_loss: 2.2700
Epoch 2/10
24/24 ━━━━━━━━━━━━━━━━━━━━ 4s 160ms/step - accuracy: 0.3701 - loss: 2.2346 - val_accuracy: 0.4796 - val_loss: 2.2000
Epoch 3/10
24/24 ━━━━━━━━━━━━━━━━━━━━ 3s 134ms/step - accuracy: 0.5240 - loss: 2.1699 - val_accuracy: 0.5953 - val_loss: 2.1338
Epoch 4/10
24/24 ━━━━━━━━━━━━━━━━━━━━ 5s 107ms/step - accuracy: 0.6207 - loss: 2.1013 - val_accuracy: 0.6455 - val_loss: 2.0599
Epoch 5/10
24/24 ━━━━━━━━━━━━━━━━━━━━ 2s 99ms/step - accuracy: 0.6404 - loss: 2.0241 - val_accuracy: 0.6728 - val_loss: 1.9763
Epoch 6/10
24/24 ━━━━━━━━━━━━━━━━━━━━ 3s 103ms/step - accuracy: 0.6667 - loss: 1.9373 - val_accuracy: 0.6833 - val_loss: 1.8835
Epoch 7/10
24/24 ━━━━━━━━━━━━━━━━━━━━ 3s 131ms/step - accuracy: 0.6783 - loss: 1.8425 - val_accuracy: 0.7013 - val_loss: 1.7836
Epoch 8/10
24/24 ━━━━━━━━━━━━━━━━━━━━ 3s 103ms/step - accuracy: 0.6864 - loss: 1.7424 - val_accuracy: 0.7172 - val_loss: 1.6804
Epoch 9/10
24/24 ━━━━━━━━━━━━━━━━━━━━ 3s 101ms/step - accuracy: 0.7123 - loss: 1.6404 - val_accuracy: 0.7327 - val_loss: 1.5769
Epoch 10/10
24/24 ━━━━━━━━━━━━━━━━━━━━ 3s 96ms/step - accuracy: 0.7236 - loss: 1.5404 - val_accuracy: 0.7433 - val_loss: 1.4771
<keras.src.callbacks.history.History object at 0x0000028F2839F6F0>
```

```python
results = model.evaluate(x_test, y_test, verbose=0)
print('Test loss, Test accuracy:', results)

plt.figure(figsize=(12, 5))

plt.subplot(1, 2, 1)
plt.plot(mod.history['accuracy'], label='Training Accuracy', color='blue')
plt.plot(mod.history['val_accuracy'],
         label='Validation Accuracy', color='orange')
plt.title('Training and Validation Accuracy', fontsize=14)
plt.xlabel('Epochs', fontsize=12)
plt.ylabel('Accuracy', fontsize=12)
plt.legend()
plt.grid(True)

plt.subplot(1, 2, 2)
plt.plot(mod.history['loss'], label='Training Loss', color='blue')
plt.plot(mod.history['val_loss'], label='Validation Loss', color='orange')
plt.title('Training and Validation Loss', fontsize=14)
plt.xlabel('Epochs', fontsize=12)
plt.ylabel('Loss', fontsize=12)
plt.legend()
plt.grid(True)

plt.suptitle("Model Training Performance", fontsize=16)
plt.tight_layout()
plt.show()
```




```python
import matplotlib.pyplot as plt

def plot_neural_network(model):
    layers_info = [
        ('Flatten', 784),
        ('Dense', 256),
        ('Dense', 128),
        ('Dense', 10),
    ]

    fig, ax = plt.subplots(figsize=(16, 10))
    ax.axis('off')

    n_layers = len(layers_info)
    x_positions = [i * 3 for i in range(n_layers)]
    colors = ['#4CAF50', '#2196F3', '#9C27B0', '#FF5722']
    n_show = 8  # nodes to show per layer

    # Draw connections first (so they appear behind nodes)
    for i in range(n_layers - 1):
        x1 = x_positions[i]
        x2 = x_positions[i + 1]
        n1 = min(layers_info[i][1], n_show)
        n2 = min(layers_info[i + 1][1], n_show)
        y1_list = [(j - n1 / 2) * 0.8 for j in range(n1)]
        y2_list = [(j - n2 / 2) * 0.8 for j in range(n2)]
        for y1 in y1_list:
            for y2 in y2_list:
                ax.plot([x1, x2], [y1, y2], 'gray', alpha=0.1, linewidth=2.5, zorder=1)
```

```
    # Draw nodes on top
    for i, (x, (name, size)) in enumerate(zip(x_positions, layers_info)):
        n_nodes = min(size, n_show)
        y_positions = [(j - n_nodes / 2) * 0.8 for j in range(n_nodes)]
        for y in y_positions:
            circle = plt.Circle((x, y), 0.15, color=colors[i], zorder=3)
            ax.add_patch(circle)

        if size > n_show:
            ax.text(x, min(y_positions) - 0.3, '...', ha='center', fontsize=12)

        ax.text(x, max(y_positions) + 0.4, f'{name}', ha='center', fontsize=11, fontweight='bold')
        ax.text(x, max(y_positions) + 0.2, f'({size})', ha='center', fontsize=9, color='gray')

    ax.set_xlim(-1, x_positions[-1] + 1)
    ax.set_ylim(-5, 5)
    plt.title('Neural Network Architecture', fontsize=15, fontweight='bold')
    plt.tight_layout()
    plt.show()

plot_neural_network(model)
```